

Midnight Griddle

1. Project Overview

Midnight Griddle is a management simulation where the player acts as the sole operator of a burger stand during the 11 PM to 6 AM shift. The core gameplay revolves around a high-pressure loop by taking orders, assembling ingredients via a drag-and-drop interface, and serving customers before their patience expires.

The game emphasizes resource management and strategic decision-making. Players must monitor limited stock levels and decide when to restock items versus when to prioritize active orders.

2. Project Review

An existing project is Papa's burgeria. A 2D restaurant management sim focused on multitasking and order accuracy.

Proposed Improvements:

Inventory management: Unlike the original's infinite supplies, players must manage a limited stock.

High Stake shift: Clear closing time (around 7 mins per night)

Money management: Players need to decide how they will restock their ingredients without making their cash flow hit 0.

3. Programming Development

3.1 Game Concept

Midnight Griddle is a high pressure, station based management simulation. Player takes on the role of a night-shift cook operating a burger stand from 11 PM to 6 AM. Unlike traditional cooking simulators, this game focuses on the anxiety of limited resources and limited customer serving window

Objectives:

Maintain Shop Reputation: Successfully fulfill customer orders to keep the shop rating above 2 stars.

Inventory management: Monitor and restock depleting ingredients (patties, buns, toppings) before they run out.

Shift Completion: Manage the workload efficiently to survive until the 6 AM clock-out.

Key Mechanics:

Station-Based Interface: The player toggles between three primary views: the Order Counter, the Grill Station, and the Assembly Table.

Drag-and-Drop Assembly: A physics-lite UI where players must manually drag ingredients to stack burgers. Accuracy in layering directly impacts the final tip and rating.

Resource Management: Ingredients are finite. The player must strategically trigger Restock Events, which cost money and take time, potentially leaving customers waiting.

Patience Decay: Each customer has a visible patience meter. If an order takes too long due to poor multitasking or lack of stock, the customer will leave, negatively impacting the shop's star rating.

3.2 Object-Oriented Programming Implementation

This subject to change for some attribute and method but core functionality of the class stay the same

GameData (Singleton)

Acts as the central State Container (The Briefcase) for the entire game session.

Key Attributes: * _money, _events, _dt, _held_item

Key Methods: * grab_item(), release_item(), update_frame_data()

GameManager

The Director of the application. Initializes the hardware, manages the main loop and handles the scaling of the game's resolution.

Key Attributes: * screen, game_wrapper, game_data, stationmanager

Key Methods: * handle_events(), update(), render(), main(), playing()

StationManager

Router between different gameplay areas. Owns swapping station UI and decides which station logic should be executed.

Key Attributes: * stations, current_station

Key Methods: * switch_station(), main()

Station (Base Class)

The blueprint for all game stations(Grill, Order, Build, restock). Handles the basic rendering of backgrounds and station objects or machines.

Key Attributes: * allitems, blocks, background

Key Methods: * handle_events, draw(), update()

InteractiveIngredient (Sprite)

Sprite objects the player can manipulate (Patties, Buns, Cheese).

Key Attributes: * dragging, rect, image

Key Methods: * update(), on_plate() <-might change later

StationBlock (Sprite) (Base class)

Represents areas or rect in a station where ingredients can be placed.

Key Attributes: * contained_items

Key Methods: * place_item

3.3 Algorithms Involved

State Management: Controls the transitions and data persistence between the Counter, Grill, and Build views.

List Matching Validation: An algorithm that compares the player's ingredient sequence against the customer's order list to calculate accuracy.

Weighted Random Spawning: Determines customer arrival frequency and surprise event triggers based on the current game hour.

I might add more in the future

4. Statistical Data (Prop Stats)

4.1 Data Features

- 1. **Net Revenue** (Money) after customer served*
- 2. **Customer Satisfaction** (Rating) after customer served*
- 3. **Customer Throughput** (number of customer) collects every 10 seconds*
- 4. **Assembly Accuracy** (Items Stacked) after finished assembling*
- 5. **Ingredients sold.** (after customer served)*

4.2 Data Recording Method

Data will be stored in **two separate CSV** files to prevent data blending.
order_log.csv will store **event based** data (Revenue, Satisfaction, Accuracy).

system_log.csv will store **time series** data (Customer amount, Cursor Movement).

4.3 Data Analysis Report

Statistical Measures:

Mean for Customer Throughput

Net Revenue (Line Graph): Displays the **Mean Revenue per Game Hour**. This identifies the most profitable periods of the night shift and analyzes how well the player manages their cash flow against restocking costs.

Customer Satisfaction (Pie Chart): Represents the **Ratio of Star Ratings** (from 1 to 5 stars). This provides a quick visual breakdown of the player's overall service quality.

Customer Throughput (Bar graph): This illustrates the density of the shift, showing when the shop was at its busiest.

Assembly Accuracy (Histogram): Shows the frequency of high-accuracy vs. low-accuracy builds. This helps determine if the drag and drop mechanics are too difficult or too easy.

Ingredients sold(Table): Rank the ingredients sold and show how much unit they are sold.

4.4 Statistical Data Revision

	Why is it good to have this data? What can it be used for	How will you obtain 100 values of this feature data?	Which variable (and which class will you collect this from?)	How will you display this feature data (via summarization statistics or via graph)?
Net Revenue	To analyze player money management and difficulty. Ex. The restock price might be too low or too high.	Data collected every time a customer is served or restock items.	Variable is money Collected from GameData	Line graph
Customer satisfaction	To analyze player time management and difficulty of multi-tasking. Ex: If satisfaction is low and customer throughput is high. We can see the game is too hard	Data collected every time a customer is served.	Variable is Rating. Class is GameData	Pie Chart
Customer throughput	To show the busyness of the shift	Data collected every 10	len(customer) From	Bar graph.

		seconds. (Amount of customer in the shop)	Customer class.	
Assembly Accuracy	Show the frequency of player accuracy. Used to analyze if order is too confusing or drag and drop is too clunky.	Data collected Every finished burger is built.	Ingredient From station class(This might be changed)	Histogram
Ingredients sold	To show and rank how much each ingredient we sold.	Data collected Every time a player serves customers.	Event from GameData class	Table that ranks and shows ingredients name, Units Used. I might also put mean and sd here as well.

4.5 Statistical Data Revision

	Feature name	Graph Objective	Graph type	X-axis	Y-axis
Line graph	Net Revenue	To visualize player total revenue	Time-series	Timestamp	Revenue

Pie Graph	Customer satisfaction	To see proportion of each rating	Proportion	Rating	-
Histogram	Assembly Accuracy	To see how order accuracy is distributed	Distribution	Timestamp	Accuracy (percentage)
Bar graph	Customer throughput	To show how many customers are in the queue or the shop.	Time-series	Hours in the game. Split each bar into half hours. Ex: 12 am, 00:30 am, 1 am.	Numbers of customer in the shop

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Changed game
3	10	Basic UI and state management for Station-based interface
4	11	Assembly mechanics. Drag and drop mechanics.
5	12	Ordering and customer system (rating included)
6	13	Implement data logging (Separate csv file for event based and time based)
7	14	Submission week (Draft)

6. Document version

Version: 1.2

Date: 28 March 2025

Editor: Thichanon Kangkachit

Date	Name	Description of Revision, Feedback, Comments
14/3	Nattanan	Fix your spelling before turning in. A basic spelling fix is already suggested by the google docs itself. Also fix the wording too. Also remove italics from your work Expand more about inherited class
21/03	Amornrit	Using bold font will improve your proposal readability. Adding key attributes and methods to each class is appreciated.
25/3	Nattanan	check out the new requirements from professor supaporn, fill that table in then resent again Add enter between each classes to make it easier to read Also when resent it is better to use the same file so i can recheck if you fix what i told you too and that will increase the speed of grading

